

Executive Summary

ARA-1 (Autonomous Research Agent): Architecture Specification

ARA-1 (Autonomous Research Agent, version 1) is a purpose-built agentic AI system designed to replicate and automate the research workflow of a junior financial analyst. Operating within the fictional quantitative research firm QuantumEdge Research, ARA-1 receives a natural language research query, independently formulates a research strategy, gathers data from multiple heterogeneous sources, such as SEC EDGAR filings, financial data APIs, earnings call transcripts, news feeds, and web search, and synthesizes all gathered information into a structured, professional-grade investment research report. It does this entirely without step-by-step human guidance.

The Problem ARA-1 Solves

The global investment research market is valued at over \$8 billion annually, yet the junior analyst workflow that underlies it remains largely manual, slow, and prone to human error. A junior analyst at an investment bank typically spends hours gathering data from multiple sources, reconciling conflicting figures, and synthesizing findings into a structured report. This process is highly repeatable and data-intensive, but requires significant coordination across tools and data sources. The core challenge is not any individual step in isolation, but the end-to-end orchestration: receiving an ambiguous query, deciding what to look for and where, resolving contradictions between sources, recovering when a data source is unavailable, and synthesizing everything into a coherent analytical narrative.

Existing LLM-powered chatbots cannot solve this problem reliably. They rely on static training data with no access to real-time financial information, cannot verify their own outputs against authoritative sources, and hallucinate confidently when they lack knowledge. ARA-1 addresses this by shifting the LLM's role from being the source of information to being the reasoning engine that orchestrates information retrieval, synthesis, and verification across a registry of 10+ external tools.

ARA-1 Key Design Decisions

ARA-1's architecture is built around five core design pillars: a ReAct reasoning loop with a maximum iteration limit and explicit termination conditions; a tool registry of 10+ tools covering SEC filings, financial data, earnings transcripts, web search, news sentiment, vector memory, fact-checking, peer comparison, financial calculation, and report generation; a three-layer memory system combining short-term context management, long-term vector storage via ChromaDB (with a Pinecone migration path), and episodic strategy logging; a multi-source synthesis engine implementing a five-tier source reliability hierarchy and a systematic conflict

resolution protocol; and a comprehensive error handling framework featuring exponential backoff retry logic, per-tool fallback chains, graceful degradation, and a circuit breaker pattern.

Agent Pattern Rationale

Two primary agent architecture patterns were considered for ARA-1: ReAct (Reasoning + Acting) and Plan-and-Execute. Both are well-established reasoning frameworks for building autonomous AI systems, and each has trade-offs. This section documents what each pattern does, the tradeoffs between them, and the reasoning behind selecting ReAct as ARA-1's core reasoning framework.

Overview of ReAct

ReAct is a reasoning framework where the agent alternates between three phases, thought, action, and observation, in a continuous loop. First, the agent generates a Thought: a reasoning step where the LLM examines everything it has gathered so far and decides what to do next. Then it takes an Action: it invokes a specific tool with specific parameters. Then it receives an Observation: the tool's output, which is added to the agent's context before the next Thought is generated. This Thought-Action-Observation cycle repeats until the agent determines it has gathered sufficient information to produce its final output, or until a maximum iteration limit is reached as a safety mechanism against infinite loops.

The defining characteristic of ReAct is that reasoning and acting are tightly interleaved. The agent does not commit to a research plan upfront. Instead, it reasons, acts, receives new information, and reasons again in direct response to what it just found. Each Observation directly shapes the next Thought, which directly shapes the next Action. The loop is continuous and adaptive by design. In many cases, ReAct is often not as efficient in token usage as it must reevaluate after each loop, making its optimal use case in static reasoning tasks.

Overview of Plan + Execute

Plan-and-Execute separates the research process into two distinct phases. A Planner LLM first reads the incoming query and generates a complete, structured research plan: a numbered sequence of steps with expected outputs and dependencies between them. Only after this plan is produced does an Executor begin working through it step by step, with optional plan revision if intermediate results require changes to the original structure.

The primary advantage of Plan-and-Execute is visibility and upfront structure. The full research plan is available for inspection before a single tool is called, which means a human reviewer can assess the research approach, identify gaps, and suggest modifications before any execution begins. For complex, well-defined research tasks where the scope of required information is

reasonably predictable in advance, Plan-and-Execute can also be more efficient than ReAct, as the structured plan reduces the likelihood of redundant or exploratory tool calls.

Reasoning for Choosing ReAct

The primary reason ReAct was selected for ARA-1 is that financial research is fundamentally non-linear, and the information discovered during research frequently changes the direction of the research itself.

Financial data is dense with surprises. Revenue figures that appear consistent on a financial data API may tell a different story when cross-referenced against a 10-K. An earnings call transcript may contradict what the most recent quarterly filing projects. A web search may surface a regulatory investigation not yet reflected in any official filing. In each of these cases, the agent cannot know in advance that this information exists, as it only becomes apparent once a tool has been called and the result returned.

Because ReAct reasons fresh after every observation, every surprising data point is immediately processed before the next action is decided. Each inconsistency, whether it be a restatement, a segment reclassification, or a divergence between sources, becomes a decision point resolved within the reasoning loop itself rather than forcing the agent to deviate from a pre-committed plan. The more financial statistics ARA-1 is processing, the more valuable this real-time adaptability becomes.

Reasoning for Not Choosing Plan-and-Execute

Plan-and-Execute's upfront visibility helps to understand the agent's methodology, allowing the analyst to make changes where necessary. However, its central limitation is that the research plan is written before any tools have been called and before any data has been retrieved. If a planned analytical framing is invalidated by data retrieved mid-execution, the agent must either continue with an incorrect plan or invoke a revision process that adds latency and failure modes. For ARA-1, which is designed to handle contradictory data and ambiguous queries across eight progressive research challenges, the adaptability of ReAct is the more important property.

Cognitive Loop Design

How the Agent Receives a Query

A research query arrives as a natural language string, such as "Produce a comprehensive risk assessment for Tesla Inc.", and is passed directly into the agent's main execution loop. No preprocessing or reformatting is applied to the query before it reaches the LLM. The raw query is included in the first user message, alongside the system prompt, which provides the agent with everything it needs to begin reasoning: its role, its tools, its constraints, and its output format requirements.

What the System Prompt Injects

The system prompt contains four components injected in sequence.

The first is the role definition, which establishes the agent as an autonomous financial research analyst operating at a quantitative research firm, with high standards for factual accuracy and source attribution.

The second is the full tool registry, injected programmatically from the tool registry module at runtime. Each tool entry includes its name, a 1-3 sentence description of when and why to use it, and its full input schema. The LLM reads these descriptions during every Thought step to determine which tool is most appropriate for the information it currently needs.

The third component is the constraint set, which governs agent behaviour throughout the session: never fabricate data, always cite sources, cross-reference numerical claims from at least two sources, report conflicting data transparently, do not make investment recommendations, and adhere to a maximum of 20 tool calls per research task.

The fourth is the output format specification, which defines the required sections of the final research report, which include Executive Summary, Company Overview, Financial Analysis, Risk Assessment, Competitive Position, and Research Methodology Notes. This ensures the agent understands what a complete output looks like before it begins gathering data.

The Thought-Action-Observation Cycle

Once the query and system prompt are passed to the LLM, the reasoning loop begins. Each iteration of the loop consists of three phases.

In the Thought phase, the LLM generates a reasoning trace in plain text. This trace reflects the agent's current understanding of the research task: what information has been gathered so far, what is still missing, which tool is most likely to provide the next needed piece of information, and whether any previously retrieved data contains inconsistencies that need to be investigated.

In the Action phase, the agent selects a tool from the registry and invokes it with a specific set of parameters derived from the Thought step. Tool calls are formatted as structured JSON objects that are parsed from the LLM's output by the agent's parser module. The parser extracts the tool name and parameter values, validates them against the tool's input schema, and dispatches the call to the appropriate tool implementation. If the LLM produces a malformed tool call, such as missing a required parameter, or invoking a tool that does not exist in the registry, the parser

catches the error and returns a structured error message as the Observation, prompting the agent to try again.

In the Observation phase, the tool's output is returned to the agent and appended to the running context. The Observation is formatted with a clear source tag, indicating which tool produced it and when, so the LLM can generate accurate citations in its final report. The agent then generates the next Thought step in direct response to what the Observation contained, and the cycle repeats.

Termination Conditions

The loop terminates under three conditions. The first is task completion: the agent determines in a Thought step that it has gathered sufficient information across all required report sections and explicitly signals readiness to generate the final output. The second is the maximum iteration limit: the agent is capped at 20 tool calls per research session, after which it is instructed to produce the best available report from the information gathered so far, with transparent notes on what could not be completed. The third is a circular reasoning detection check: if the agent invokes the same tool with the same parameters more than twice within a single session, the loop flags this as circular reasoning, logs the pattern, and redirects the agent to either try an alternative tool or proceed with available information.

Handling Ambiguous Queries

ARA-1 handles ambiguity through a disambiguation step that runs before the main research loop begins.

When the agent's query analyser classifies an incoming query as ambiguous based on the absence of a named entity, a specific time period, or a clear analytical objective, the agent generates a set of documented assumptions rather than requesting clarification. These assumptions are logged at the top of the final report under the Research Methodology Notes section, so the reader understands the interpretive choices the agent made. For example, faced with "What's happening with the banks?", the agent might assume the query refers to major US commercial banks, interpret "happening" as referring to the most recent quarter's financial results and any material news events from the past 30 days, and proceed on that basis. If a query is sufficiently specific that disambiguation is unnecessary, this step is skipped, and the main loop begins immediately.

Memory Architecture

Short-Term Memory (Working Memory)

A primary risk with short-term memory is overflow, which ARA-1 manages through progressive summarisation: when the running context approaches 70% of the model's context window limit, the earliest Thought-Action-Observation traces are compressed into a structured summary that retains key findings, numerical claims, source attributions, and identified conflicts, while discarding the raw tool output text. This summary is prepended to the active context so the agent retains awareness of earlier research steps without the full token cost.

Long-Term Memory (Vector Database)

Long-term memory persists across all eight research challenges and is implemented using ChromaDB for development, with a migration path to Pinecone for production deployment. When a research session completes, key findings are chunked, embedded using OpenAI's text-embedding-3-small model, and stored using a metadata schema including ticker symbol, source type, date, confidence score, and a verified flag indicating whether the claim was cross-referenced against multiple sources using the fact-checker tool.

The chunking strategy varies by document type. SEC filings are chunked by section, with Risk Factors, MD&A, and Financial Statements each treated as discrete retrieval units. Earnings transcripts are chunked by speaker turn, keeping each Q&A pair together to preserve the context of analyst questions alongside management responses. News articles are chunked by paragraph, with the headline always included as context. Financial statement data is stored as structured JSON and retrieved via metadata filtering rather than semantic search.

Episodic Memory (Strategy Memory)

After each completed research challenge, the agent logs which tool sequences produced high-confidence results, which fallback chains were invoked and whether they succeeded, and which query types triggered the failure modes including circular reasoning, premature conclusion, and context overflow. These logs are stored in a structured JSON format and are summarised into the system prompt at the start of future sessions as lightweight strategic guidance.

Tool Registry Specification

Functional Categories and Tools

ARA-1's tool registry contains 12 tools organised across five functional categories. Each tool is defined by a JSON schema following the OpenAI function calling specification, which is a natural language description.

Data Retrieval: sec-filing-search retrieves 10-K, 10-Q, 8-K, and DEF 14A filings from SEC EDGAR by ticker and filing type, forming the Tier 1 highest-reliability source in the synthesis engine's source hierarchy. financial-data-api retrieves structured income statement, balance sheet, cash flow, and ratio data from Financial Modeling Prep. earnings-transcript retrieves quarterly earnings call transcripts with speaker labels. company-profile retrieves basic company metadata, including sector, market cap, and executive roster.

Search and Sentiment: web-search performs real-time web searches for news, commentary, and recent developments using the Tavily API. news-sentiment analyses sentiment across recent news articles using NLP scoring.

Memory: vector-db-search queries the ChromaDB long-term vector database for previously researched information. vector-db-store stores new research findings with the structured metadata schema.

Analysis: peer-comparison identifies industry peers and retrieves comparative financial metrics, designated as an expected tool for cloud computing division comparison across AWS, Azure, and GCP. calculation-engine performs financial calculations including DCF valuation, growth rates, margins, and ratios, unlocked after the progression unlock system introduces more advanced tools. fact-checker cross-references specific numerical claims against multiple authoritative sources and is designated as the primary verification tool for contradictory data investigations.

Output: report-generator formats all gathered and synthesised findings into a structured markdown research report following the template sections: Executive Summary, Company Overview, Financial Analysis, Risk Assessment, Competitive Position, and Research Methodology Notes.

Tool Selection Logic and Anti-Patterns

The LLM selects tools based on the descriptions in the registry and the current state of the research. Before invoking any external tool, the agent checks whether the needed information already exists in the current context or can be retrieved from long-term memory via vector-db-search. This guards against the most costly anti-pattern: making redundant external API calls for information the agent already holds in context or has previously stored.

Additional anti-patterns explicitly guarded against include over-reliance on web-search for structured numerical data that is more accurately available from financial-data-api or sec-filing-search; invoking financial-data-api for a single ratio when a full statement retrieval would provide multiple needed data points in one call; and proceeding to report generation

without invoking fact-checker for critical numerical claims, which previously produced a 23% hallucination rate in the Alpha Research Labs case study before their redesign.

Every primary tool has a defined fallback sequence. If sec-filing-search fails, the agent falls back to web-search targeting SEC.gov directly and then to vector-db-search for previously stored filing excerpts. If financial-data-api fails, the system falls back to sec-filing-search with manual extraction and then to web-search targeting financial summary platforms. If earnings-transcript retrieval fails, the fallback mechanism shifts to web-search targeting earnings coverage from major financial publications.

Fallback Chains

All fallback invocations are logged with reduced confidence flags that are surfaced in the final report's Research Methodology Notes section to maintain transparency when degraded retrieval occurs.

RAG Pipeline Design

ARA-1's RAG pipeline follows a six-stage architecture designed to prevent hallucinations, particularly the generation of plausible but incorrect financial figures that are not grounded in source documents.

Stage 1: Query Transformation. The raw user query is decomposed into three to five retrieval queries optimized for different source types. For example, a Tesla risk assessment query may generate separate retrieval requests targeting SEC risk disclosures, recent news on emerging risks, earnings call commentary on risk management, and analyst sentiment.

Stage 2: Multi-Source Retrieval. These retrieval queries are sent simultaneously to the long-term vector database, SEC EDGAR, the Financial Modeling Prep API, the Tavily news API, and the earnings transcript database. Since each source returns data in different formats, all outputs are normalized into a common document schema before moving to the next stage. This ensures the report incorporates multiple source types.

Stage 3: Relevance Filtering and Re-Ranking. Retrieved documents are scored against the original query using a cross-encoder re-ranking model. Documents below the relevance threshold are discarded. Domain-specific re-ranking ensures financially equivalent concepts are treated as semantically related, such as recognizing that "EBITDA margin expansion" is highly relevant to a query about operating margins.

Stage 4: Context Assembly. Retrieved documents are assembled into the LLM's context window using a fixed token allocation strategy. Primary data sources such as filings and financial statements receive the largest allocation, followed by supporting evidence such as news and analyst commentary. Space is also reserved for system instructions and final generation. Each document chunk is tagged with source information, date, and reliability tier to support accurate citations.

Stage 5: Grounded Generation. The LLM generates responses using only the assembled context and follows strict constraints. It cannot fabricate data, must cite every factual claim, must cross-reference numerical information across multiple sources, and must clearly separate factual evidence from inference.

Stage 6: Post-Generation Verification. A separate LLM call verifies every numerical claim against retrieved source documents. Claims that cannot be traced back to a source are flagged as hallucinations and are either corrected or explicitly labeled as uncertain. If sources disagree by more than 5%, the discrepancy is documented in methodology notes.

Error Handling Framework

ARA-1's error handling framework addresses common financial research failures through retry logic, fallback systems, and graceful degradation. This is treated as a core architectural requirement because inaccurate financial reports can lead to material financial consequences.

Retry Logic

All external API calls are wrapped using the Python library tenacity. The retry system begins with a one-second delay and doubles after each attempt, up to five retries. Each retry includes random jitter to prevent simultaneous repeated requests from overwhelming external services.

Transient failures such as timeouts, HTTP 503 errors, and rate-limit rejections trigger retries automatically. Permanent failures such as authentication errors and malformed responses bypass retries and immediately move to fallback systems. This distinction is critical because APIs such as Alpha Vantage and NewsAPI enforce strict daily request limits.

Fallback Chains and Graceful Degradation

When retries fail, fallback tools are invoked. The system logs the original error, fallback tool used, and reduced confidence level. If all fallback options fail, the report continues using available information while explicitly documenting missing data. The system never fabricates missing information.

Circuit Breaker

A circuit breaker prevents cascading failures. If a tool fails more than three times during a research session, the system stops calling it and routes future requests directly to fallback tools. This prevents a single failing API from blocking the entire workflow.

Synthesis Engine Design

ARA-1's synthesis engine is responsible for combining multiple sources into coherent analysis and is one of the most difficult parts of the architecture.

Source Reliability Hierarchy

The system prioritizes SEC filings such as 10-Ks, 10-Qs, 8-Ks, and DEF 14A filings because they are audited and legally regulated. Financial data APIs are treated as the next most reliable source, followed by earnings call transcripts. Major news outlets such as Reuters, Bloomberg News, and Financial Times are used after that. Social media and unverified forums are considered the least reliable and are used only for sentiment analysis, never for numerical claims.

Conflict Resolution and Synthesis Techniques

When sources report conflicting values for the same metric, the system first checks whether the discrepancy is caused by different reporting periods. It then checks whether financial statements were restated. If the conflict remains unresolved, the higher-reliability source is prioritized.

Narrative threading connects findings from multiple sources into a coherent story, while quantitative triangulation compares the same metric across multiple independent sources.

Evaluation Framework Overview

ARA-1 evaluates performance using more than twenty quality metrics across factual accuracy, completeness, analytical depth, coherence, and agent behavior. Performance is benchmarked against reports produced by firms such as Goldman Sachs, Morgan Stanley, and Gartner.

The framework measures numerical accuracy, citation quality, temporal accuracy, entity accuracy, hallucination rates, section coverage, source diversity, historical coverage, risk factor coverage, insight density, cross-source synthesis, original calculations, forward-looking analysis, logical flow, internal consistency, formatting quality, tool efficiency, error recovery rates, memory utilization, planning quality, and end-to-end latency.

Quantitative metrics are evaluated automatically, while qualitative metrics are assessed using LLM-as-judge evaluation frameworks.

Technology Stack and Environment

ARA-1 is built in Python using LangChain as the primary agent framework and LangGraph for stateful orchestration.

LangGraph manages the ReAct workflow as a directed graph with conditional transitions and enables dynamic tool expansion without requiring changes to the agent core.

The reasoning engine uses Anthropic's Claude Sonnet as the primary model because of its strong structured output and tool usage capabilities. Its 200K token context window reduces context overflow during long research sessions. OpenAI's GPT-4o serves as a fallback model.

The vector database uses Chroma during development and includes a migration path to Pinecone Systems for production deployment. Embeddings are generated using OpenAI's text-embedding-3-small model with 1,536 dimensions.

External integrations include SEC EDGAR, Financial Modeling Prep, Tavily, and NewsAPI. Centralized rate limiting and response caching prevent API budget exhaustion across repeated research workflows.